

Sensor fault detection using machine learning

TOMÁŠ TICHÝ

Department of Electrical Engineering
Czech Technical University
tichyt11@fel.cvut.cz

Abstract—This work deals with detection of various sensor faults with a neural network. A classifier network based on the LSTM architecture is proposed for drift, offset and outlier error detection. Temperature sensor data from an intelligent building are first augmented with artificially generated error signals and then used for both the neural network training and evaluation.

Index Terms—sensor fault detection, machine learning, neural network, LSTM.

I. INTRODUCTION

INTELLIGENT houses are becoming more popular, thanks to the convenience and energy savings they offer. The systems involved rely heavily on large amounts of data from multiple sensors. These sensors are however prone to many types of errors, which can make system behave suboptimally. An effective method to automatically detect these errors is therefore crucial for smooth operation of such systems. A popular approach includes training a neural network to detect or even diagnose the error using the raw sensor data. The detection of an error can be solved for example with time-series anomaly detection using a neural network auto-encoder [4] that evaluates its reconstruction error to detect an anomaly. The diagnosis of the fault has to be solved using a classifier network [1], that tries to directly predict the type of error. The goal of this work is to design, implement and train a neural network classifier to detect various types of faults in intelligent building sensors.

A. Problem specification

The proposed neural network architecture will be both trained and evaluated on available multi-sensor data from a family house. The available dataset consists of two sensor types (temperature and humidity sensors) placed in different places around a family house. It includes a total of 14416 sensor readings for each of the 19 sensors (10 temperature sensors and 9 humidity sensors). The sensor data are in the form of multiple time-series, one for each sensor, sampled once per hour.

The main assumption made here is that the individual time-series should be correlated and thus a sensor fault can be inferred from the difference in behaviour between the sensors. For sake of simplifying the problem, only the temperature sensor data will be used. Using all the sensors at once would unnecessarily lead to a larger neural network and therefore longer training times. The available sensor data are assumed to be fault-free and therefore the erroneous data for training and evaluation has to be generated artificially.

1) *Assumptions*: The sensor error can be of three types: drift, offset and outlier values. Furthermore, these types of error can all happen at the same time, as can be the case in real data. To further simplify the problem, only one of the sensors can be faulty at any given time and the other sensors are assumed to be functional. Since the goal is to detect an error, the output of the network should be binary (true/false) for each signal error type. [6]

II. METHODS

A. Data preprocessing

1) *Splitting into segments*: Let the vector

$$\mathbf{s}[t] = [s_1[t] \quad s_2[t] \quad \dots \quad s_k[t]]^T \quad (1)$$

denote the measurements of all the k sensors at time $t \in \{1, 2, \dots, N\}$, where N is the number of measurements. The sensor data can be thought of as a sequence of such measurements $\mathbf{S}_{data} = (\mathbf{s}[1], \mathbf{s}[2], \dots, \mathbf{s}[N])$. The $\mathbf{S}_{data}$ is first split up into shorter overlapping time segments

$$\mathbf{S}_\tau = (\mathbf{s}[1 + \tau], \mathbf{s}[2 + \tau], \dots, \mathbf{s}[n + \tau]) , \quad (2)$$

$$\tau \in \{0, 1, \dots, N - n\}$$

of length n , that are better suited for neural network training and inference. The segment length n should be chosen to be long enough to enable reliable fault detection. On the other hand, longer segment length also leads to longer delay between fault and its detection.

2) *Error generation*: Given a segment of measurements $\mathbf{S} = (\mathbf{s}[d])_{d=1}^n$ of length n (here d marks the delay with respect to the segment), label the measurements of the first sensor as $s_1 = (s_1[d])_{d=1}^n$. Then the error $e = (e[d])_{d=1}^n$ can be applied to the original sequence s_1 additively as $s_e[d] = s_1[d] + e[d]$, generating faulty sensor measurements $s_e = (s_e[d])_{d=1}^n$. The faulty data segment $\mathbf{S}_f$ is then constructed by replacing s_1 by s_e in $\mathbf{S}$.

As already discussed, three types of error are modeled here: drift, offset and outliers. The drift is modeled as a linear function of $(d - 1)$, the offset as a constant function of d and lastly the outlier error is modeled as a single pulse at $d = d_0$, where d_0 is drawn uniformly from $\{2, 3, \dots, n - 1\}$. This can be summarized as

$$\begin{aligned} e_d[d] &= p \cdot (d - 1) , \\ e_{off}[d] &= q , \\ e_{out}[d] &= r \cdot \delta_{dd_0} , \end{aligned} \quad (3)$$

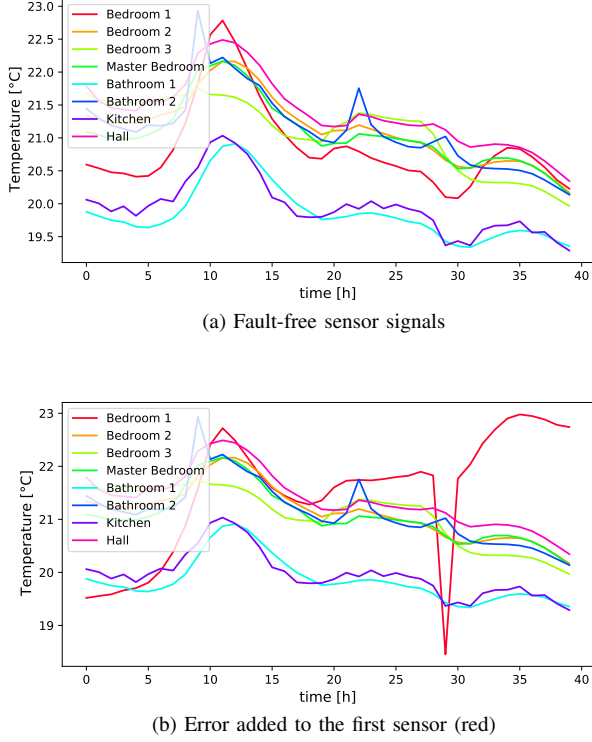


Fig. 1. Data segment.

where e_d , e_{off} and e_{out} are the drift, offset and outlier error sequences respectively and δ_{dd_0} denotes the Kronecker delta¹. The parameters

$$\begin{aligned} p &\sim U([-p_{max}, -p_{min}] \cup [p_{min}, p_{max}])^2, \\ q &\sim U([-q_{max}, -q_{min}] \cup [q_{min}, q_{max}]), \\ r &\sim U([-r_{max}, -r_{min}] \cup [r_{min}, r_{max}]) \end{aligned} \quad (4)$$

are generated randomly for each data segment. In this way, they are guaranteed to be non-zero and to have some minimum magnitude as a zero magnitude would mean there is no error. Generating errors randomly ensures variability in the data and prevents overfitting of the network. The combined errors can be generated simply by adding the relevant error functions together, since they are additive.

It is not clear how to choose the intervals for the error generation ($p_{min}, \dots, r_{max}$). This is because multiple factors affect how the temperatures in the individual locations evolve and thus the fault-free data already include some amount of variability between the sensors. Therefore the error intervals have been heuristically tuned, in order for the errors to be distinguishable enough in the data and yet to not be too easy for the network to detect. An example of the sensor signals for segment length $n = 40$ can be seen in figure 1a. The figure 1b then shows the data with drift, offset and outlier errors added to the first signal.

¹ $\delta_{ij} = 1$ for $i = j$ and $\delta_{ij} = 0$ for $i \neq j$

² $x \sim U(A) \iff x$ is uniformly distributed on A

B. Neural network architecture

1) *Reccurent neural network*: For time-series data it is favourable to use a recurrent neural network (RNN), which is a type of a neural network, that is able to process temporal information. It processes the whole time-series sequentially, one timestep at a time, while also taking into account its state from the previous timestep. However, ordinary recurrent networks suffer from a vanishing gradient problem [3] and as such struggle with long time sequences. A great enhancement can be made by using a Long Short Term Memory [6] (LSTM) network. A single layer LSTM network performs the following computation

$$\begin{aligned} i[t] &= \sigma(W_{ii}x[t] + b_{ii} + W_{hi}h[t-1] + b_{hi}) \\ f[t] &= \sigma(W_{if}x[t] + b_{if} + W_{hf}h[t-1] + b_{hf}) \\ g[t] &= \tanh(W_{ig}x[t] + b_{ig} + W_{hg}h[t-1] + b_{hg}) \\ o[t] &= \sigma(W_{io}x[t] + b_{io} + W_{ho}h[t-1] + b_{ho}) \\ c[t] &= f[t] * c[t-1] + i[t] * g[t] \\ h[t] &= o[t] * \tanh(c[t]) \end{aligned} \quad (5)$$

at every time step t of the input sequence $x[t]$. The σ in equations (5) is the sigmoid function³ and $*$ denotes the Hadamard product⁴. The W and b coefficients denote the weights and biases, which are the learnable parameters of the network. The $c[t]$ and $h[t]$ are the cell state and hidden state at time t respectively. The cell state is what gives LSTM its advantage over an ordinary RNN. It enables the network to store information over larger time spans, which helps alleviate the vanishing gradient problem.

2) *Inputs and Outputs*: The goal of the neural network is to take a given sequence of measurements $\mathbf{S} = (\mathbf{s}[d])_{d=1}^n$ and detect possible faults of the first sensor signal $s_1 = (s_1[d])_{d=1}^n$. There are in total 7 possible combinations of the 3 errors (eq. (3)). Together with the fault-free case, there are 8 classes that the signal could belong to. Therefore the neural network should be able to produce at least 8 distinct outputs. Instead of the network output being 8 probabilities for every possibility, it is a better idea to separate the problem into 3 decision problems for each error. The network should therefore map a sequence of measurements $\mathbf{S} = (\mathbf{s}[d])_{d=1}^n$ to a vector of 3 probabilities $[p_1, p_2, p_3]^T$, one for each error type. The probabilities can then be thresholded to get the final predicted label (true/false) for each error type.

3) *The proposed network*: The proposed neural network (fig. 2a) can be divided into two parts: The recurrent stage and the fully connected stage. In the recurrent stage (fig. 2b), the measurements $\mathbf{s}[1]$ are used to initialize the hidden state h_0 and cell state c_0 as

$$\begin{aligned} h_0 &= \tanh(W_h \mathbf{s}[1] + b_h), \\ c_0 &= \tanh(W_c \mathbf{s}[1] + b_c), \end{aligned} \quad (6)$$

where W and b are the learnable weights and biases. Then the LSTM network processes the whole sequence of measurements (eq. 5) $\mathbf{S}$ and outputs the last hidden state $h[n]$ to the

³ $\sigma(x) = \frac{e^x}{e^x + 1}$

⁴An element-wise product

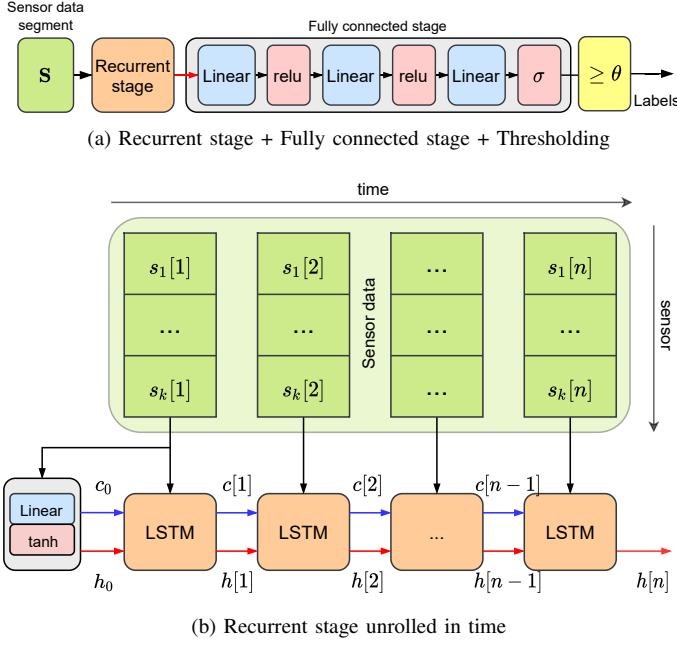


Fig. 2. Proposed classification algorithm

fully connected stage. The fully connected stage is a simple network made up of two linear layers ($y = Wx + b$) with rectified linear function activation⁵ and one linear layer with sigmoid activation function. The sigmoid activation function ensures, that the output values are between 0 and 1. These outputs can then be thresholded by the value $\theta \in [0, 1]$ to get the predicted true/false labels for each error type.

C. Training

As already discussed, the training data are split into smaller time segments $\mathbf{S} = (\mathbf{s}[d])_{d=1}^n$, on which the inference is performed. Each time segment $\mathbf{S}$ is then used to create 7 faulty time segments $\{\mathbf{S}_f^1, \dots, \mathbf{S}_f^7\}$, with every possible combination of the modelled errors. The target labels (desired outputs of the network) are of the form $l = [l_1, l_2, l_3]^T$, where $l_1, l_2, l_3 \in \{0, 1\}$. Each entry in the label corresponds to a type of error. If the error is present in the data, the corresponding label should be 1 and 0 otherwise. The order of the labels can be chosen arbitrarily, but here the order $[l_d, l_{off}, l_{out}]^T$ for drift, offset and outlier label respectively is used (e.g. the label $[1, 0, 1]^T$ means there are drift and outlier errors present, but no offset). The prediction of these 3 labels can be understood as 3 distinct binary classification problems. Therefore a binary cross entropy loss should be used for the back-propagation. The binary cross entropy loss for a predicted probability $p_i \in [0, 1]$ and a target label $l_i \in \{0, 1\}$ can be computed as

$$BCE L_i = -[l_i \cdot \log(p_i) + (1 - l_i) \cdot \log(1 - p_i)]. \quad (7)$$

This loss is then averaged over all the predictions. However, because of convergence issues, a mean square error loss can be used for initial training session and to be later replaced by

⁵ $y = x$ for $x > 0$ and $y = 0$ for $x \leq 0$

TABLE I
COMPARISON OF DIFFERENT SEGMENT LENGTHS⁶

n	15	20	25	30	35	40
drift acc.	73.8%	82.1%	94.5%	95.7%	98%	96%
offset acc.	77.9%	84.4%	88.7%	88.1%	93.3%	85.1%
outlier acc.	95.7%	99.9%	96.6%	99.6%	91.7%	99.7%

the BCE loss for fine tuning. The mean square error loss can be computed as the average of squared error

$$SE = (l_i - p_i)^2, \quad (8)$$

over all predictions. In either case, the loss is minimized using Adam [2] optimizer.

D. Implementation details

As per the actual implementation, the python library PyTorch [5] was used for neural network architecture and training. The size of the hidden layers was tweaked numerous times and settled on the values $[20, 32, 32]$ for the LSTM hidden layer and the first and second linear layers in the fully connected stage respectively. The raw dataset of 14416 sensor readings has been split up into an evaluation dataset of 1000 readings used for evaluation during training (hyperparameter tuning), a verification dataset of 1000 readings used for result testing and a training dataset. The last two temperature sensors located in the attic and outside of the building are less correlated with the rest of the sensors and have not been used at all. The parameter intervals for the generation of errors have been heuristically set to $[p_{min}, p_{max}] = [0.075, 0.1]$, $[q_{min}, q_{max}] = [1, 2]$ and $[r_{min}, r_{max}] = [2.5, 3.5]$ for the drift, offset and outlier faults respectively. The in-built PyTorch Adam optimizer has been used with parameters $weight_decay = 10^{-4}$ and $eps = 10^{-4}$.

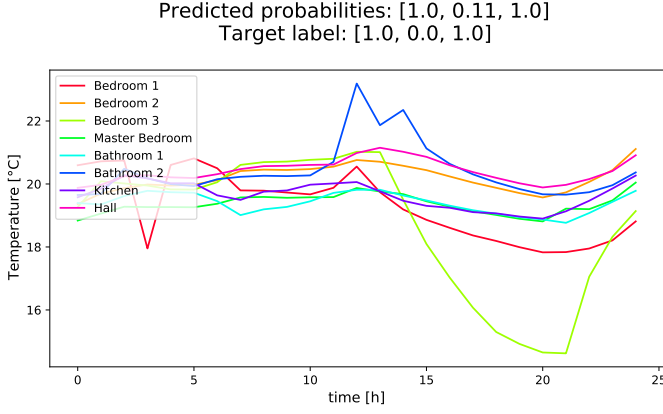
III. RESULTS

The proposed network has been trained with learning rates $lr \in [10^{-5}, 10^{-2}]$ until convergence to some reasonable local minima of the evaluation loss. In the longer segments $n \geq 40$, the model training failed to converge with the BCE loss and therefore the MSE loss was used in few thousand initial training epochs.

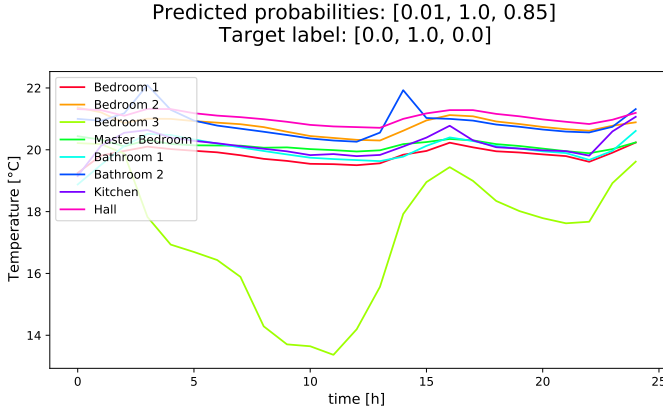
A. Effects of sequence length on prediction accuracy

The table I shows the percentage of correct predictions on the verification dataset with respect to the different time segment lengths n used for training and inference. The drift, offset and outlier accuracies show the percentages of correct predictions. The predicted probabilities were thresholded by 0.5 in this case. So if for the i -th error the predicted probability is $p_i \geq 0.5$, then the predicted label is 1, and if $p_i < 0.5$ then the predicted label is 0. The figure 3 shows a pair of prediction examples for segment length $n = 25$, where the first one 3a includes a drift error hardly visible from the data

⁶Results obtained with $\theta = 0.5$



(a) Example of a correct prediction



(b) Example of a wrong prediction

Fig. 3. Examples of network predictions for $n = 25$

and yet the network predicts it correctly. The second example in fig. 3b on the other hand predicts an outlier error, although there is no clear sign of one.

B. Effects of thresholding value

The output of the proposed neural network consists of three probabilities, each of which estimates, how probable the given error type is, given the data segment provided. By thresholding them, the true/false label can be obtained. However, the thresholding value θ does not need to be exactly 0.5, but instead can be larger to reduce the number of false positives⁷, or smaller to reduce the number of false negatives⁸. The table II shows the effect of choosing different thresholding values θ on the false positive (FP) and false negative (FN) percentages. These results were obtained with a trained neural network with input segment length $n = 35$. Different thresholds $[\theta_d, \theta_{off}, \theta_{out}]$ can also be used for each error, which enables tweaking FP percentages for each one individually. For example, setting $\theta_d = \theta_{off} = 0.999$ and $\theta_{out} = 0.9$, would yield (based on results in II) false positive percentages of 0%, 0.03% and 0% for the drift, offset and outlier errors respectively, making false alarms rare.

⁷False positive $\equiv$ error has been detected, even though there is none

⁸False negative $\equiv$ error has not been detected, even though there is one

⁹Results obtained with $n = 35$

TABLE II
COMPARISON OF DIFFERENT THRESHOLDING VALUES⁹

θ	Drift		Offset		Outlier	
	FP [%]	FN [%]	FP [%]	FN [%]	FP [%]	FN [%]
0.001	99.3	0	99.9	0	54.2	0.1
0.1	8.9	0.7	24.6	2.6	5.9	4.4
0.5	2.3	1.7	6	7.5	1.05	15.6
0.9	0.9	5.1	1.8	18.8	0	50.6
0.999	0	58.6	0.0	82.2	0	100

IV. CONCLUSION

A. Summary

The available dataset of family house temperature readings has been split into smaller time segments more suitable for use with a neural network. Three types of errors: drift, offset and outliers, have been generated based on some assumptions and added to the original fault-free data to create a training dataset for the neural network. A suitable neural network classifier based on the LSTM architecture has been proposed, trained and evaluated on the created dataset.

B. Discussion

The results (table I) have shown that with longer time segment lengths used for training and inference the drift accuracy in general rises with segment length as does the offset. The outlier accuracy seems to be largely unaffected by the segment length. The model trained on segment length $n = 35$ seems to be the most accurate, since it reached the highest drift and offset accuracies, although the outlier accuracy was quite low. Lower n could be of use, if low detection delay is of the essence, while losing some accuracy on the drift and offset error detection.

The peak drift and outlier accuracies (98%, resp. 99.9%) are fairly reasonable results, considering the network has not seen the verification data during training. This means that it learned to generalize the concept of these two errors quite well. Tuning the threshold values θ (as discussed in III-B) could further improve the results in terms of false positive or negatives and make up an effective error detection tool.

The offset error on the other hand peaked at only 93.3%, which is a lot worse. However, this relatively low accuracy might have been caused by the heuristically generated offset error being too small to be reliably detected when compared to the natural variance in the data itself. This problem stems from the fact that it is hard to generate error randomly, while making sure it produces meaningful results (is detectable but not too easy to detect) in every time segment of the dataset. One possible solution would be to generate the error not completely randomly, but using some knowledge about the particular time segment of data (e.g. max, min, average values, etc.). But the best solution would be to simply gather an actual dataset of real-world faulty sensor data and use that for training and evaluation.

REFERENCES

- [1] Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Deep learning for time series classification: a review. *Data Mining and Knowledge Discovery*, 33(4):917–963, 2019.
- [2] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 12 2014.
- [3] John F. Kolen and Stefan C. Kremer. *Gradient Flow in Recurrent Nets: The Difficulty of Learning LongTerm Dependencies*, pages 237–243. 2001.
- [4] Jinman Park. RNN based Time-series Anomaly Detector Model Implemented in Pytorch. <https://github.com/chickenbestlover/RNN-Time-series-Anomaly-Detection>, 2018. Accessed: [Insert date here].
- [5] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- [6] Hasim Sak, Andrew W. Senior, and Françoise Beaufays. Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition. *ArXiv*, abs/1402.1128, 2014.